

COLLEGE EVENT MANAGEMENT SYSTEM

*A Project Report submitted in partial fulfillment of the requirements for the
award of degree of*

BACHELOR OF TECHNOLOGY
in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

Shaik Afeef (24B11CS513)

V Jaya Sai Krishna (24B11CS514)

Mahammad Hussain Sharif (24B11CS517)

Ibanoshi Kariwo (24B11CS542)

Under the esteemed guidance of

Dr. Hari Jyothula,

Assistant Professor.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

2022-2026

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work entitled “**COLLEGE EVENT MANAGEMENT SYSTEM**” is being submitted by

Shaik Afeef (24B11CS513)

V Jaya Sai Krishna (24B11CS514)

Mahammad Hussain Sharif (24B11CS517)

Ibanoshi Kariwo (24B11CS542)

in partial fulfillment of the requirements for award of the B.Tech. degree in Computer Science and Engineering. This project has been carried out under the guidance of Dr. Hari Jyothula, Assistant Professor, Department of CSE , Aditya University. We certify that this is a genuine work by the students and has not been submitted previously anywhere for any degree.

Project Guide

Dr. Hari Jyothula,
Associate Professor,
Department of CSE

Head of the Department

Dr. T Sudha Rani,M.Tech.,Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the project entitled “**COLLEGE EVENT MANAGEMENT SYSTEM**” is a genuine project. This work has been submitted to **ADITYA UNIVERSITY** (Formerly Aditya Engineering College (A)), Surampalem in partial fulfilment of the requirements for the B.Tech. degree. We further declare that this project work has not been submitted in full or in part for the award of any degree or any other educational institutions.

By

Shaik Afeef (24B11CS513)

V Jaya Sai Krishna (24B11CS514)

Mahammad Hussain Sharif (24B11CS517)

Ibanoshi Kariwo (24B11CS542)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. J Hari, Associate Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K. Chandra Sekhar, Associate Professor, & Dr. Hari Jyothula, Associate Professor**, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. T Sudha Rani, Associate Professor**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of Contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1-2
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3-4
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	3-4
2.3 Requirements Specification	4
3. TECHNOLOGY DESCRIPTION	5-6
3.1 Frontend Technologies	5
3.2 Backend Technologies	5
3.3 Database Technology	6
3.4 Tools & IDEs	6
4. DATABASE DESIGN	7-9
4.1 Database Collections (MongoDB)	7
4.2 Schema Structures	7-8

4.3 Keys and Constraints	8-9
4.4 Collection Relationships	9
5. SYSTEM DESIGN	10-11
5.1 System Architecture	10
5.2 Module Description	10-11
6. SYSTEM IMPLEMENTATION	12-16
7. CONCLUSION	17
8. FUTURE ENHANCEMENTS	18
9. REFERENCES	19
 APPENDIX	 20-21
A.1 – Sample Code	20-21
A.2 – MongoDB Query Examples	21

ABSTRACT

The College Event Management System (CEMS) is a full-stack web application designed to streamline the management of academic, cultural, and sports events within a university campus. The system addresses the inefficiencies of manual, paper-based event coordination by providing a centralized digital platform that connects three types of users: Administrators, Organizers, and Students.

Built using the MERN stack (MongoDB, Express.js, React.js, Node.js), the application supports the complete event lifecycle — from event creation and admin approval to student registration and participant management. Its Role-Based Access Control (RBAC) model ensures data security and user-specific dashboards.

The system is fully deployed and accessible at: <https://college-event-managementsystem-1-gbfo.onrender.com>

LIST OF FIGURES

Figure 1: Login/Registration Page

Figure 2: Student Dashboard – Event Feed

Figure 3: Organizer Dashboard – Event Management

Figure 4: Admin Dashboard – Analytics Panel

Figure 5: Event Creation Form

Figure 6: Event Approval Workflow (Admin)

Figure 7: Participant Management (Organizer)

Figure 8: System Architecture Diagram

Figure 9: Entity-Relationship (ER) Diagram

Figure 10: CSV Export – Admin Report

LIST OF TABLES

Table 1: Student Collection Schema

Table 2: Organizer Collection Schema

Table 3: Event Collection Schema

Table 4: Registration Collection Schema

Table 5: Admin Collection Schema

Table 6: ActivityLog Collection Schema

Table 7: Feedback Collection Schema

Table 8: API Endpoints Summary

ACRONYMS

CEMS : College Event Management System

MERN : MongoDB, Express.js, React.js, Node.js

API : Application Programming Interface

REST : Representational State Transfer

RBAC : Role-Based Access Control

GUI : Graphical User Interface

CRUD : Create, Read, Update, Delete

ODM : Object Document Mapper

SPA : Single Page Application

CSE : Computer Science and Engineering

JSON : JavaScript Object Notation

CSV : Comma-Separated Values

URL : Uniform Resource Locator

UI : User Interface

UX : User Experience

1. INTRODUCTION

1.1 Brief Information about the Project

The College Event Management System (CEMS) is a comprehensive web-based platform developed to digitize and automate the management of university events. Universities organize numerous events throughout the academic year — technical symposiums, cultural fests, sports tournaments, workshops, and seminars. Coordinating these events manually creates significant administrative burden, frequent data errors, and poor student participation due to lack of awareness.

CEMS provides an integrated platform for three key stakeholders: the Administrator who governs the system, Organizers who create and manage events, and Students who discover and register for events. Each user type has a dedicated, role-specific dashboard built using React.js, connecting securely to a Node.js/Express.js backend and a MongoDB Atlas cloud database.

1.2 Motivation and Contribution of Project

In many universities, event management is still handled through circular notices, physical registration counters, and manually maintained spreadsheets. This leads to several key problems:

- Redundant data entry across different event coordinators
- Difficulty in tracking student participation and event attendance
- Delays in obtaining event approvals from administrators
- Lack of centralized visibility into ongoing and upcoming events
- Inability to generate real-time analytics or participant rosters

CEMS directly addresses these pain points by providing a single, secure, always available web platform. It reduces manual effort, eliminates paper-based processes, and ensures accurate, real-time data for all stakeholders involved in college event coordination.

1.3 Objectives of the Project

The main objectives of the College Event Management System are:

- To develop a role-based web application for managing all types of college events
- To allow organizers to submit event proposals for admin review and approval
- To enable students to browse, filter, and register for approved events online
- To provide administrators with live analytics (total students, organizers, pending approvals)
- To implement secure database storage of all event, user, and registration data
- To support CSV export of participant and student reports for record-keeping
- To deploy the system to a publicly accessible cloud environment

1.4 Scope of the Project

The College Event Management System is designed to serve:

- University administration offices
- Student councils and event organizing committees
- Faculty coordinators managing department-level events
- Individual students wanting to discover and participate in campus events

The system supports the full event lifecycle — from proposal to completion — with role-specific dashboards accessible via any modern web browser. The deployed version at Render ensures 24/7 availability for all registered users.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations the College Event Management System must perform:

User Registration and Login: The system shall allow students and organizers to register with unique IDs and emails, and log in with password authentication.

Role-Based Routing: The system shall redirect users to role-specific dashboards (Admin, Organizer, or Student) upon successful authentication.

Event Creation: The system shall allow organizers to create events with title, date, time, location, category, description, price, max participants, and poster.

Event Approval: The system shall require admin approval before an event is visible to students.

Event Browsing and Filtering: The system shall allow students to browse and filter all admin-approved events by category.

Event Registration: The system shall allow students to register for events individually or as a team, subject to availability.

Participant Management: The system shall allow organizers to view, filter, and export participant rosters for their events.

Admin User Management: The system shall allow the admin to view all students and organizers, and delete accounts if necessary.

Dashboard Analytics: The system shall provide the admin with counts of total students, organizers, events, and pending approvals.

CSV Export: The system shall allow admins and organizers to export data reports in CSV format.

Feedback Submission: The system shall allow students to submit event feedback with a rating and comment.

Password Recovery: The system shall allow users to reset passwords using a security question-and-answer mechanism.

2.2 Non-Functional Requirements

Non-functional requirements define the quality standards the system must meet:

Performance: API responses for common operations (login, event listing) shall complete within 2 seconds under normal network conditions.

Security: The system shall enforce role-based access, preventing unauthorized users from accessing protected routes or data.

Usability: The UI shall be intuitive, modern, and accessible via standard desktop and tablet browsers without any training.

Reliability: The system shall handle data submission errors gracefully, displaying meaningful error messages to users.

Scalability: The MongoDB Atlas cloud database shall scale to handle a growing number of events and user registrations.

Availability: The system shall remain available 24/7 through Render cloud deployment.

Maintainability: The codebase shall follow clear separation of concerns (routes, models, frontend components) for easy maintenance.

2.3 Requirements Specification

This section specifies the hardware and software requirements to run CEMS.

2.3.1 Minimum Hardware Requirements

Processor: Intel i3 or above (or any equivalent ARM/x86 processor)

RAM: 4 GB minimum (8 GB recommended for development)

Hard Disk: 500 MB free space for local installation

Internet Connection: Required for MongoDB Atlas and deployment access

Monitor: Standard display with resolution 1280×720 or higher

2.3.2 Software Requirements

Operating System: Windows 10/11, macOS, or any Linux distribution

Runtime: Node.js v18+ (for backend execution)

Package Manager: npm (Node Package Manager)

Frontend Build Tool: Vite (React.js)

Database: MongoDB Atlas (Cloud) or local MongoDB

Browser: Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions)

IDE/Editor: Visual Studio Code

API Testing: Postman or similar REST client

3. TECHNOLOGY DESCRIPTION

3.1 Frontend Technologies

3.1.1 *React.js*

React.js is an open-source JavaScript library developed by Meta (Facebook) for building fast and interactive user interfaces. It follows a component-based architecture where the UI is broken into reusable, independent components. React uses a virtual DOM to optimize rendering performance.

In CEMS, React.js powers the entire frontend — including the Login page, Student Dashboard, Organizer Dashboard, and Admin Dashboard. React Router DOM is used for client-side navigation and role-based routing.

3.1.2 *Vite (Build Tool)*

Vite is a modern frontend build tool that provides an extremely fast development server using native ES modules. Unlike older tools like Webpack, Vite serves source files over native ESM, enabling near-instant hot module replacement (HMR). In CEMS, Vite is used to bundle the React frontend for both development and production deployment.

3.2 Backend Technologies

3.2.1 *Node.js*

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It allows JavaScript to be run on the server side, enabling event-driven, non-blocking I/O operations. In CEMS, Node.js serves as the server runtime, powering all backend API operations.

3.2.2 *Express.js*

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications. It simplifies request routing, middleware integration, and HTTP utility methods. In CEMS, Express.js manages all RESTful API routes including authentication, event management, registration, and CSV export endpoints.

3.3 Database Technology

3.3.1 MongoDB & Mongoose ODM

MongoDB is a cross-platform, document-oriented NoSQL database that stores data in flexible, JSON-like documents called BSON. Unlike relational databases, MongoDB does not enforce a rigid schema, making it highly scalable and adaptable for evolving application models.

Mongoose is an ODM (Object Document Mapper) for Node.js that provides schemabased modeling for MongoDB documents. In CEMS, Mongoose defines strict schemas for each collection (Student, Organizer, Event, Registration, Admin, Feedback, ActivityLog) and handles all database operations.

3.3.2 MongoDB Atlas (Cloud)

MongoDB Atlas is a fully-managed cloud database service provided by MongoDB Inc. It hosts the CEMS database on the cloud, offering high availability, automated backups, and global accessibility. The connection is established via a secure connection URI stored in the .env environment file on the server.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code (VS Code) is the primary IDE used for CEMS development. It provides IntelliSense, integrated Git control, an extensions marketplace, and an integrated terminal — making it ideal for full-stack JavaScript/Node.js development.

3.4.2 Postman & Render

Postman was used throughout development for testing and validating all REST API endpoints before frontend integration. Render is the cloud platform used to deploy both the Node.js backend and the React frontend in a single unified service, providing HTTPS and a public URL.

4. DATABASE DESIGN

4.1 Database Collections (MongoDB)

The CEMS MongoDB database (named "collegeEvents") contains the following collections, each with a well-defined Mongoose schema:

The Student collection stores registered student profiles including credentials and event references.

The Organizer collection stores credential and identity information for event coordinators.

The Event collection stores all details for each college event, including approval status.

The Registration collection links students to events they have signed up for, with payment/status data.

The Admin collection stores administrator login credentials.

The Feedback collection stores event ratings and comments from students.

The ActivityLog collection records system actions for audit and monitoring.

4.2 Schema Structures

The following tables describe the key fields in each collection schema.

4.2.1 Student Collection

Table 1: Student Collection Schema

Field	Type	Constraint	Description
name	String	Required	Student's full name
studentId	String	Required, Unique	University ID number
email	String	Required, Unique	Registered email address
branch	String	Optional	Academic branch/department
password	String	Required	Account password (min 8 chars)
securityQuestion	String	Optional	Question for password recovery
securityAnswer	String	Optional	Answer for password recovery
registeredEvents	ObjectId[]	Reference: Event	List of event IDs registered

4.2.2 Event Collection

Table 3: Event Collection Schema

Field	Type	Constraint	Description
title	String	Required	Title of the event
date	String	Required	Date of the event
time	String	Optional	Time of the event
location	String	Optional	Venue of the event
description	String	Optional	Event details and rules
category	String	Optional	Technical / Cultural / Sports
organizerId	String	Optional	ID of the organizer who created it
price	Number	Default: 0	Registration fee (0 = free)
maxParticipants	Number	Default: null	Participation capacity limit
status	String (enum)	upcoming/ongoing/completed	Current event status
isApproved	Boolean	Default: false	Approval flag set by Admin

4.3 Keys and Constraints

MongoDB _id fields act as primary keys. Consistency across documents is maintained via:

Unique Indexes: studentId, email in Students; organizerId, email in Organizers.

Required Fields: name, password, studentId/organizerId, title/date in Event.

Enum Constraints: Event status (upcoming, ongoing, completed),
Registration paymentStatus (pending, paid, failed, free), Registration status (registered, attended, cancelled).

Reference Keys: eventId in Registration references Event._id; studentId links Registration to Student.

Default Values: price defaults to 0, isApproved defaults to false, registeredCount defaults to 0.

4.4 Collection Relationships

The collections are related as follows:

- One Student can have many Registrations (One-to-Many).
- One Event can have many Registrations (One-to-Many).
- Each Registration links exactly one Student to one Event.
- One Organizer can create many Events (One-to-Many).
- One Student can submit many Feedback records for different events (One-toMany).

This normalized design prevents data duplication and allows flexible querying using Mongoose's `.populate()` method to resolve `ObjectId` references at query time.

5. SYSTEM DESIGN

5.1 System Architecture

The College Event Management System follows a three-layer (3-tier) client-server architecture:

Presentation Layer (Client Tier): Built with React.js and Vite. The SPA serves as the browser-based UI through which all users (Admin, Organizer, Student) interact with the system via their respective role-specific dashboards.

Application Layer (Server Tier): Built with Node.js and Express.js. All business logic, authentication, event CRUD operations, registration management, and CSV export functionality are handled by the REST API server running on this layer.

Data Layer (Database Tier): MongoDB Atlas cloud database stores all persisted data across 7 collections. Mongoose ODM ensures structured interactions between the application layer and the database.

This architecture ensures separation of concerns, security, and scalability. The client communicates with the server exclusively via RESTful HTTP API calls using the Fetch API.

5.2 Module Description

The system is composed of the following key modules:

1. Authentication Module

This module handles all login and registration logic. Users provide credentials (studentId/email + password). The server searches Admin, Organizer, and Student collections in sequence and returns the user role on match. React Router then redirects to the appropriate dashboard. A security question mechanism enables password recovery.

2. Event Management Module

This module allows Organizers to create, edit, and delete events via a detailed form. New events are saved with `isApproved: false` and go into a pending queue. Admins can approve or reject events. Only approved events appear in the Student event feed. The module also tracks registration counts and capacity limits.

3. Student Module

This module provides students with a dynamic, filterable event feed. Students can register for events (individually or as a team), view their registration history, and submit feedback. A digital profile view shows their enrolled events and attendance status.

4. Admin Module

The Admin module provides centralized system control. It includes dashboard analytics (student count, organizer count, pending approvals), event moderation (approve/reject), user management (view and delete accounts), system activity logs, and the ability to export the full student database as a CSV report.

5. Organizer Module

This module allows organizers to monitor their created events, view registered participants, update attendance status, and export participant rosters as CSV files. A stats panel shows total events created and total registrations across all their events.

6. Feedback & Contact Module

This module allows students to submit post-event feedback with a star rating and textual comment. A contact form allows any user to send inquiries, stored in the Contact collection with a timestamp.

7. Activity Logging Module

This module silently records key user actions (event creation, registration, status updates) into the ActivityLog collection. Admins can view an audit trail from their dashboard.

6. SYSTEM IMPLEMENTATION

Login / Registration Page:

The authentication page is the entry point of the system. Users enter their ID/email and password. The system checks credentials against Admin, Organizer, and Student collections in order. Upon success, the user is redirected to their role-specific dashboard. Failed logins display an appropriate error message.

Figure 1: Login/Registration Page of CEMS

The figure displays two screenshots of the Aditya University login and registration interface. The top screenshot shows the login page with fields for 'User ID or Email' and 'Password', a 'Sign In' button, and links for 'Forgot Password?', 'Student Signup', and 'Organizer Signup'. The bottom screenshot shows the student registration page with fields for 'Full Name', 'Student ID', 'Branch', 'Email Address', 'Password', 'Confirm Password', 'Security Question', and 'Answer', along with a 'Create Student Account' button and a link to 'Sign In' if the user already has an account.

Aditya University
Welcome, please enter your login credentials.

User ID or Email

Password

[Forgot Password?](#)

Sign In

Don't have an account?

[Student Signup](#) [Organizer Signup](#)

[Need help? Contact Support](#)

COLLEGE EVENT MANAGEMENT PORTAL

Aditya University
Student Registration

Full Name

Student ID

Enter your full name

Enter your Student ID

Branch

-- Select Branch --

Email Address

student@college.edu

Password

Confirm Password

Enter password

Re-enter password

Password must be at least 8 characters long and include uppercase, lowercase, number, and special character.

Security Question

Answer

-- Select a question --

Enter your answer

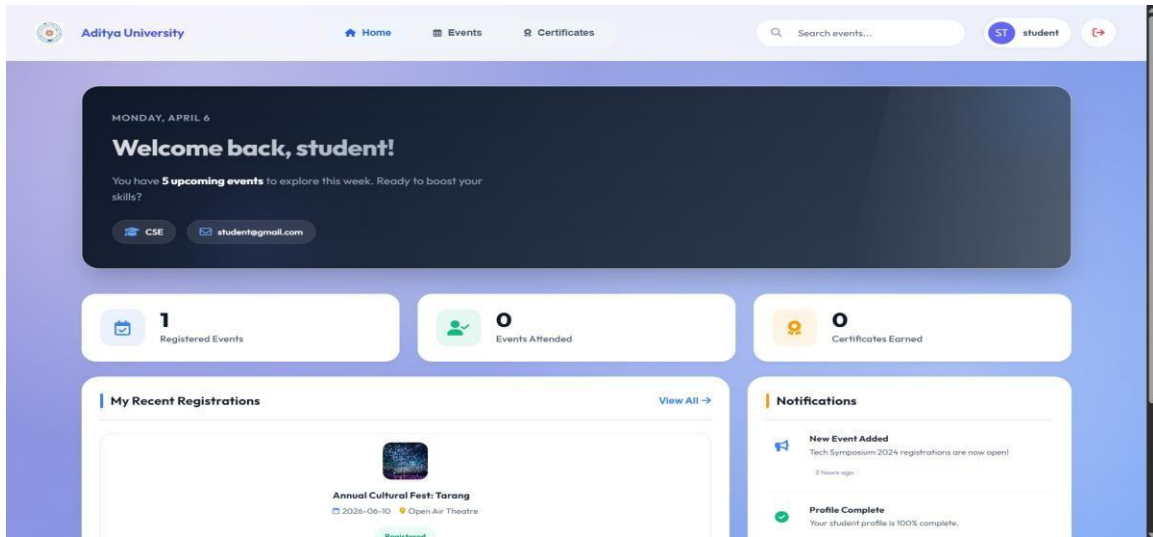
Create Student Account

Already have an account? [Sign In](#)

Student Dashboard – Event Feed:

The Student Dashboard displays all admin-approved events in a dynamic cardbased layout. Students can filter events by category (Technical, Cultural, Sports) and search by keyword. Clicking on an event card reveals full details and a registration button.

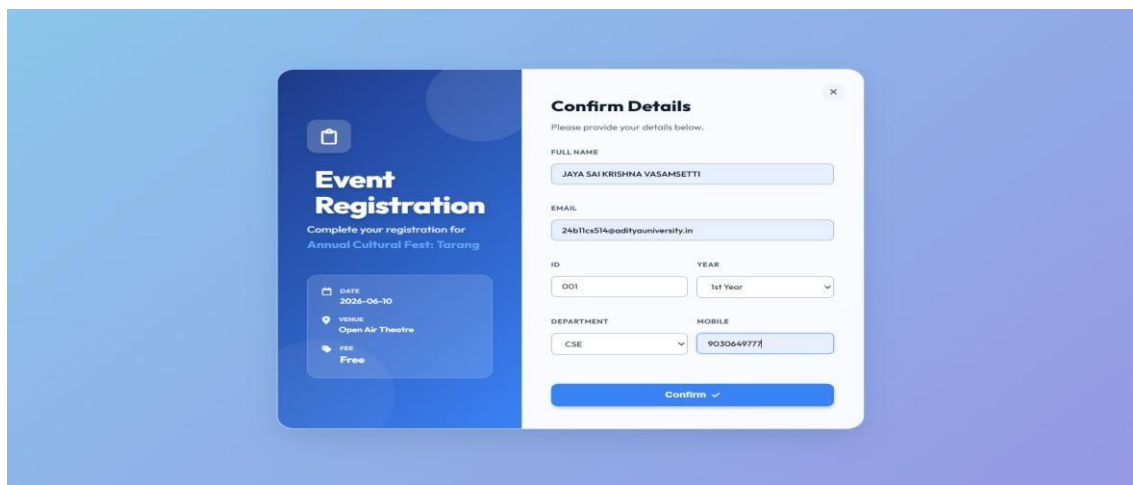
Figure 2: Student Dashboard – Event Feed



Event Registration:

When a student clicks "Register" on an event, a confirmation modal captures registration type (individual or team). The system validates capacity limits and duplicate registrations before creating a Registration document and incrementing the event's registeredCount.

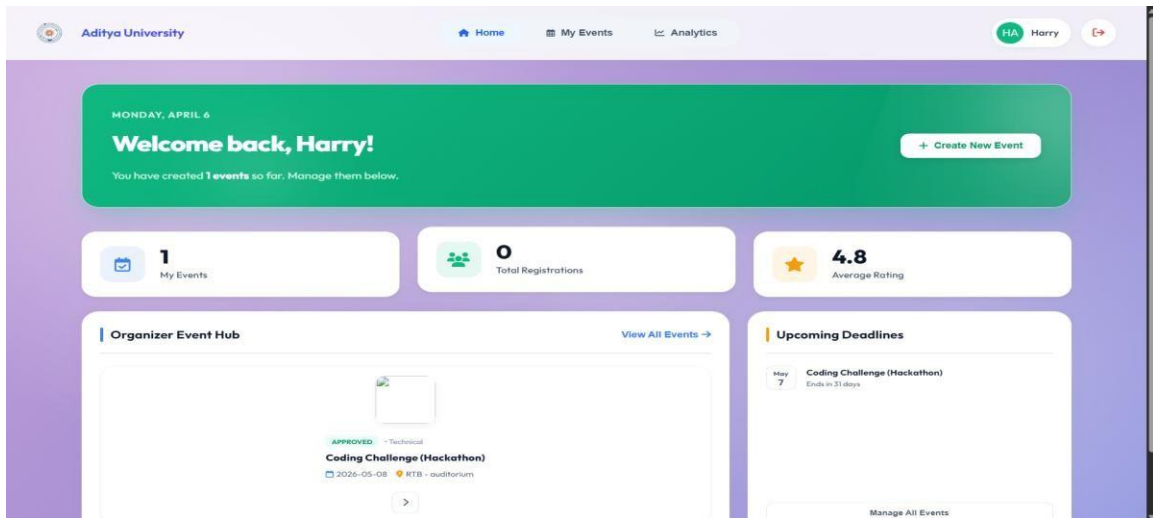
Figure 3: Event Registration Modal



Organizer Dashboard – Event Management:

The Organizer Dashboard displays the organizer's created events with their current approval status. Organizers can create new events via a detailed multi-field form, edit existing events, and delete events. A statistics card shows their total events and total registrations.

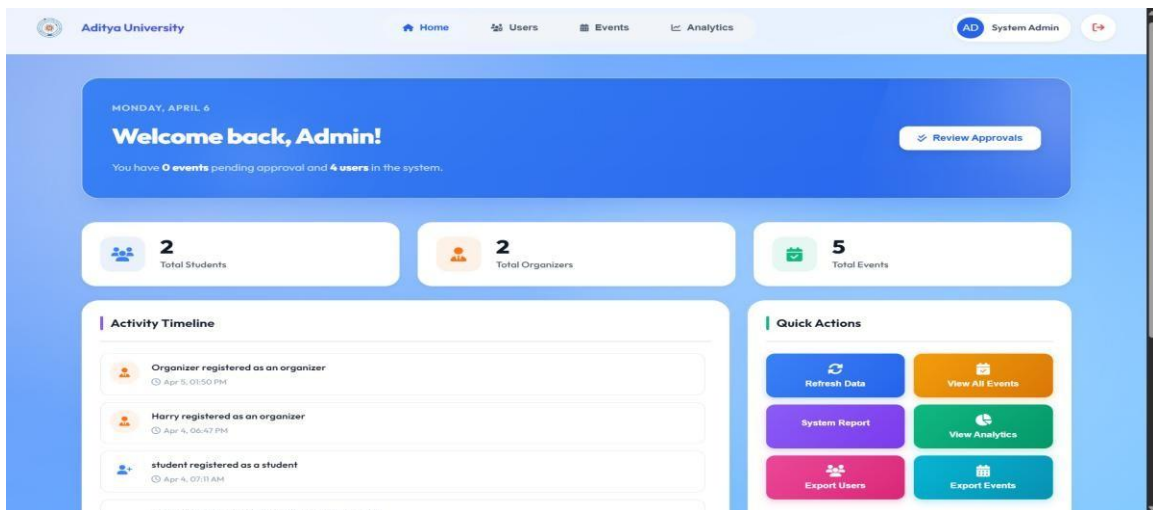
Figure 4: Organizer Dashboard – Event Management Panel



Admin Dashboard – Analytics Panel:

The Admin Dashboard provides a bird's-eye view of the entire system. Real-time counters show total students, organizers, total events, and events pending approval. The admin can switch between views for event moderation, user management, and system activity logs.

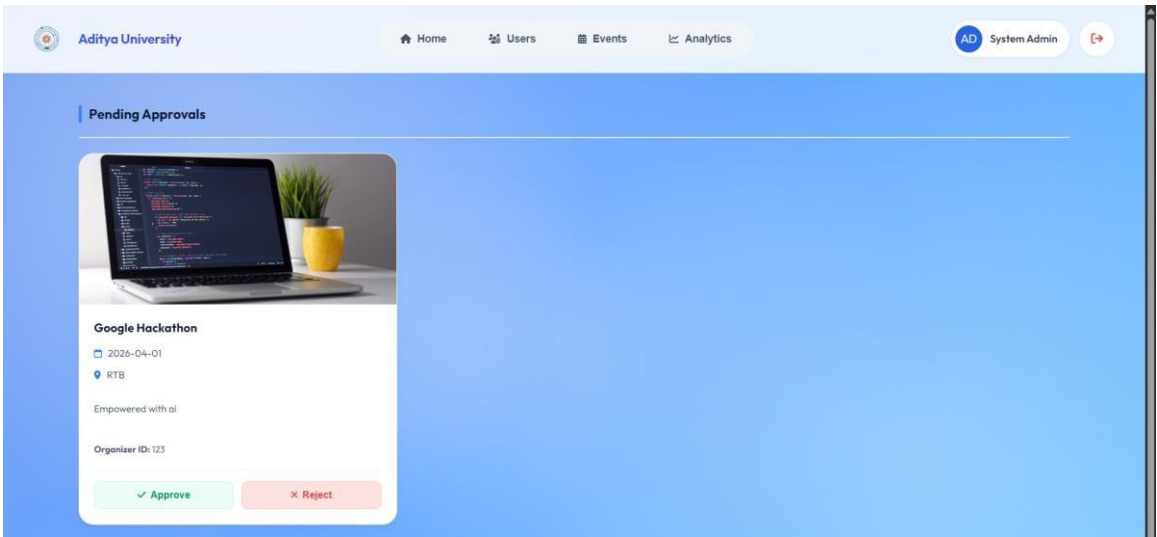
Figure 5: Admin Dashboard – Analytics Panel



Event Approval Interface (Admin):

A dedicated event queue in the Admin panel shows all events with isApproved: false. For each pending event, the admin can Approve (sets isApproved: true and makes it visible to students) or Reject (removes it from the queue).

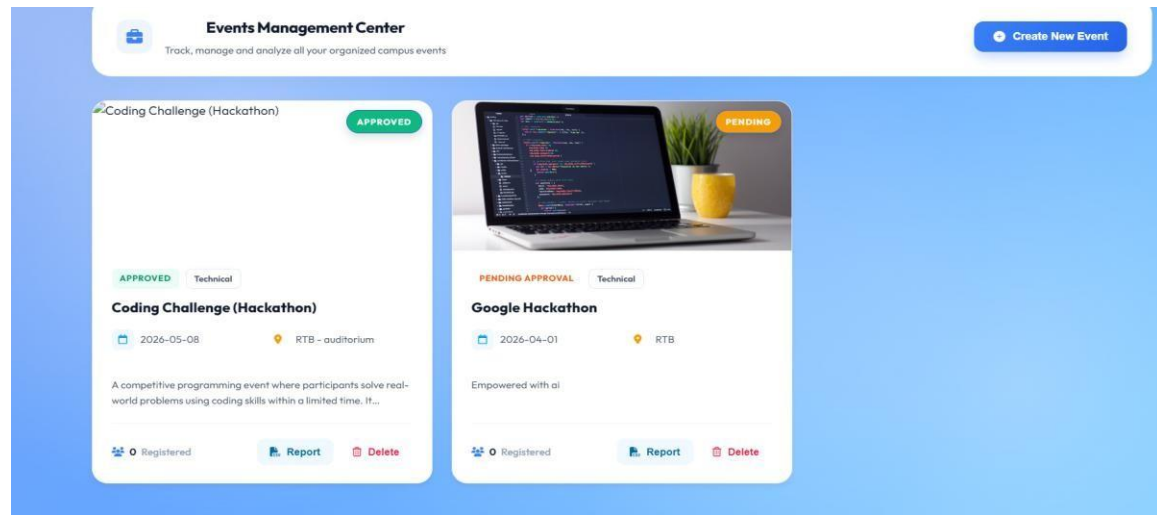
Figure 6: Admin Event Approval Interface



Participant Management (Organizer):

Organizers can open any of their events to view the full participant list. Each row shows the student name, email, branch, registration type, and status. Organizers can change attendance status and download the full list as a formatted CSV report.

Figure 7: Organizer Participant Management Table



AutoSave OFF participants_Coding_Challenge_(Hackathon) (2... Search

File Home Insert Page Layout Formulas Data Review View Help

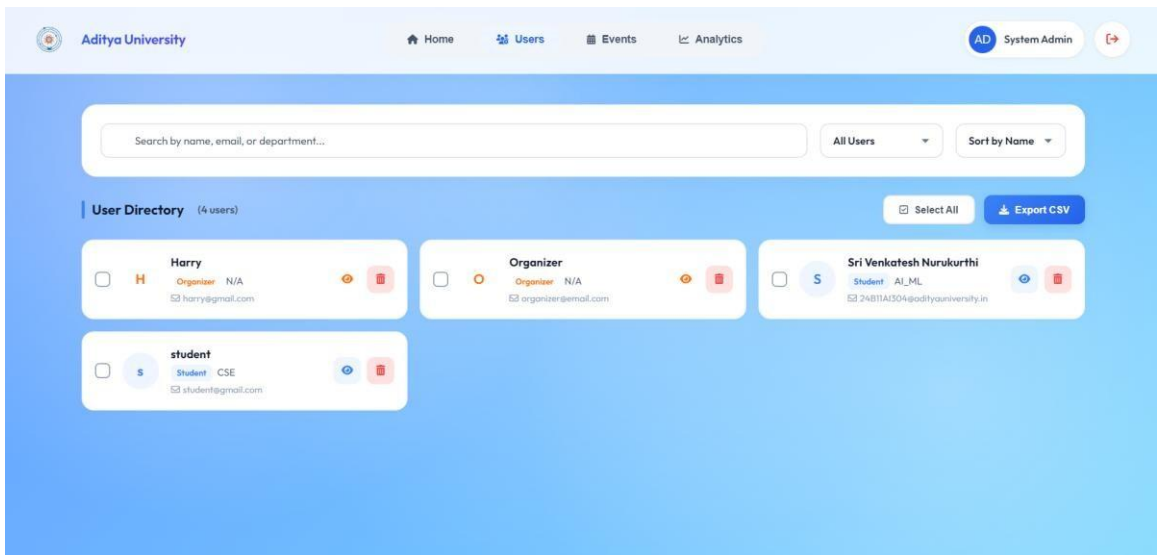
POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format. Don't show again Save As...

Event Report: Coding Challenge (Hackathon)							
RegistrationID	Student ID	Name	Email	Branch	Type	Team Name	Status
123	101	Krishna	krishna@gmail.com	CSE	tech	TechTitans	Paid

Admin User Management:

The Admin's user management view lists all registered students and organizers in a searchable table. The admin can view details and permanently delete an account. Deleting a student also cleans up their associated registration records.

Figure 8: Admin User Management Interface



7. CONCLUSION

The College Event Management System has been successfully developed and deployed as a full-stack web application using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system fulfills its core objective of digitizing the college event lifecycle from organizer event submission to student registration and administrator oversight.

The application provides a clean separation of roles through its RBAC model, ensuring that every user type (Admin, Organizer, Student) sees only the data and functionality relevant to them. The use of MongoDB Atlas as a cloud database ensures data reliability and availability.

By automating repetitive manual processes such as event registration, participant tracking, and report generation, the system significantly reduces the administrative workload on college event coordinators while improving the student experience through real-time event discovery.

The system is publicly deployed at <https://college-event-management-system-1gbfo.onrender.com> and has been verified to function correctly across all core user workflows. All project review objectives, including the implementation of backend APIs, MongoDB CRUD operations, role-based authentication, and frontend-backend integration exceeding 80% functionality, have been achieved.

Overall, the College Event Management System demonstrates the practical application of modern web development technologies in solving a real-world institutional challenge and serves as an effective, scalable foundation for future enhancements.

8. FUTURE ENHANCEMENTS

The current version of the College Event Management System provides a solid foundation for institutional event coordination. The following enhancements are planned for future versions:

Integrated Payment Gateway: Adding Razorpay or Stripe integration to process registration fees securely for premium technical workshops and seminars.

QR Code Check-In System: Generating unique QR codes upon registration for students to scan at event entry points, automating attendance verification.

Mobile Application Port: Building a React Native version of the student-facing interface to provide native iOS and Android applications.

Real-Time Notifications: Integrating WebSockets or push notification services to send students reminders about upcoming events, venue changes, or approval updates.

Advanced Analytics Dashboard: Providing organizers and admins with data visualizations (charts, graphs) showing registration trends, category popularity, and student engagement over time.

Email Notifications: Sending automated confirmation emails to students upon successful registration and to organizers when an event is approved or rejected.

Medical Records Integration: Extending the platform to track student health requirements or emergency contact info for event-specific needs (e.g., sports events).

Multi-College Support: Expanding the system to support inter-college events and external participant registrations.

Admin Report Generation: Implementing a server-side report generation engine to produce detailed PDF reports on event performance and participation statistics.

Improved UI/UX Design: Enhancing the interface with modern UI component libraries and advanced micro-animations for better user engagement.

These enhancements can make the system more scalable, intelligent, and suitable for real-time institutional use at a larger scale.

9. REFERENCES

The following resources were referenced during the development of the College Event Management System:

1. **Mardan Azat, Azat Mardan**, "Pro Express.js: Master Express.js: The Node.js Framework For Your Web Development", Apress, 2014.
2. **Chris Minnick, Eva Holland**, "Coding with JavaScript For Dummies", Wiley, 2015.
3. **Accomazzo Anthony, Murray Nate**, "Fullstack React: The Complete Guide to ReactJS", Fullstack.io, 2020.
4. **MongoDB Documentation**, <https://www.mongodb.com/docs/>
5. **Mongoose ODM Documentation**, <https://mongoosejs.com/docs/>
6. **Express.js Documentation**, <https://expressjs.com/>
7. **React Documentation (Official)**, <https://react.dev/>
8. **Vite Build Tool Documentation**, <https://vitejs.dev/>
9. **Node.js Official Documentation**, <https://nodejs.org/docs/>
10. **Render Deployment Guide**, <https://render.com/docs/>

APPENDIX

A.1 Sample Code

Database Connection Code (server.js extract):

The following code connects the Node.js server to MongoDB Atlas using Mongoose:

```
const mongoose = require('mongoose');
require('dotenv').config();

mongoose.connect(process.env.MONGO_URI ||
'mongodb://127.0.0.1:27017/collegeEvents', {  serverApi: { version: '1', strict:
true, deprecationErrors: true } })
  .then(() => { console.log('MongoDB Connected'); })
  .catch(err => console.error('Connection Error:', err));
```

Event Registration Endpoint (server.js extract):

This endpoint handles student registration for an event:

```
app.post('/api/events/register', async (req, res) => {  const { studentId,
eventId, type, teamName, teamMembers } = req.body;  const student
= await Student.findOne({ studentId });  const event = await
Event.findById(eventId);  if (!student || !event) return
res.status(404).json({ message: 'Not found' });
```

```

    if (event.maxParticipants && event.registeredCount >=
event.maxParticipants)    return res.status(400).json({ message:
'Event is full' });

    const existingReg = await Registration.findOne({ studentId, eventId
});    if (existingReg) return res.status(400).json({ message: 'Already
registered' });    const newReg = new Registration({ studentId, eventId,
registrationType: type });    await newReg.save();
event.registeredCount += 1;    await event.save();    res.json({ message:
'Successfully registered!', registrationId: newReg._id });
});

```

A.2 MongoDB Query Examples

Get All Approved Events:

```
db.events.find({ isApproved: true })
```

Get Event Participants:

```
db.registrations.find({ eventId: ObjectId('...') })
```

Count Pending Events:

```
db.events.countDocuments({ isApproved: false })
```

Delete Student Account:

```
db.students.deleteOne({ studentId: 'STU001' })
```